FoodStore - Trabajo práctico Unidad 7

Bases de Datos II - NoSQL

Integrantes: Franco Sampieri, Mauro Di Pietro, Tomas Martinez y Renzo Victoria

Repositorio: https://github.com/francosampieri/TP2_BDD2_FoodStore

1. Modelo y migración

En PostgreSQL, `pedido` tiene una relación 1:N con `detalle_pedido`. Cada pedido pertenece a `usuario`, y cada línea referencia a `producto`. Para MongoDB creé la base `foodstore_nosql` y la colección `pedidos`. Cada documento conserva `pedido.id_pedido` como `_id`, además de `fecha`, `estado`, `total`, un objeto `cliente` y un arreglo `items` con producto, cantidad, precio unitario histórico y subtotal.

Tomé cinco pedidos reales de `foodstore` (IDs 2, 3, 6, 11 y 12) y sus ocho líneas. Los importé desde `mongodb/pedidos_u7.json` mediante MongoDB Compass. Compass informó **5 documentos importados** y la colección mostró los cinco IDs. Los importes aparecen como `Decimal128`; `fecha` se guardó como texto `AAAA-MM-DD` porque en PostgreSQL es `DATE` y no incluye hora. El objeto `cliente` conserva `id_usuario` y usa un alias sintético: no se copiaron datos personales.

Embebí las líneas porque la lectura frecuente de un pedido completo obtiene cabecera y productos juntos. Conservé el precio unitario de la venta, aunque el precio actual del producto cambie.

Dos reglas que deben mantenerse:

1. `cantidad` debe ser positiva y cada `subtotal` debe coincidir con `cantidad × precio_unitario`.
2. `total` debe coincidir con la suma de los subtotales de sus líneas; al consultar ventas históricas se deben usar los precios guardados en el pedido.

Los cinco documentos cumplen ambas comprobaciones. En esta práctica no configuré un validador de esquema en MongoDB: la consistencia se verificó durante la preparación e importación de la muestra, no queda garantizada para futuras escrituras.

2. Consultas y comparación

Ejecuté las cuatro consultas en el shell integrado de Compass. En DBeaver ejecuté las dos equivalentes de PostgreSQL sobre `foodstore`, restringidas a los mismos cinco `id_pedido`. Los archivos adjuntos son `mongodb/consultas_mongodb.js` y `sql/consultas_postgresql_u7.sql`.

Consulta MongoDB	Resultado observado
Todos los pedidos (`_id`, `estado`)	2 TERMINADO; 3 TERMINADO; 6 CANCELADO; 11 CONFIRMADO; 12 TERMINADO
Estado `TERMINADO`	2, 3, 12
Línea con `id_producto = 9`	3, 12
`total > 6000.00`	2, 12

Elegí 6000,00 como umbral para separar los dos pedidos de mayor importe dentro de esta muestra. La comparación se hizo por identificador, no solo por cantidad, y no mostró diferencias:

Filtro	PostgreSQL: cantidad e IDs	MongoDB: cantidad e IDs	¿Coinciden?
Estado `TERMINADO`	3: 2, 3, 12	3: 2, 3, 12	Sí
Producto 9	2: 3, 12	2: 3, 12	Sí

Salida abreviada de ejecución (10/11/2026): Compass mostró `terminados: [{_id:2},{_id:3},{_id:12}]`, `producto9: [{_id:3},{_id:12}]` y `mayores6000: [{_id:2},{_id:12}]`. DBeaver, conectado a `foodstore`, devolvió `terminados = 2,3,12` y `producto_9 = 3,12`. En SQL apliqué `eliminado = FALSE` y `DISTINCT` para obtener un resultado por pedido.

3. Evaluación de la tecnología e IA

Herramienta consultada: Codex (OpenAI). **Consulta realizada:** «Compara PostgreSQL y MongoDB para FoodStore. El modelo actual guarda pedido, detalle_pedido, usuario y producto en PostgreSQL con claves foráneas y un procedimiento para crear pedidos. Se leen pedidos completos con frecuencia y deben mantenerse coherentes las cantidades, los precios históricos, el total y el stock. ¿Qué aporta guardar las líneas embebidas en un documento y qué operaciones seguirían necesitando control transaccional? Da afirmaciones comprobables».

Respuesta resumida de la IA: (1) Embebiendo las líneas se puede leer y actualizar el pedido completo como un solo documento; eso simplifica ese patrón de acceso. (2) Si la creación del pedido modifica también el stock de productos, una operación sobre el documento del pedido no alcanza por sí sola para hacer atómicos esos cambios en varios documentos. Recomendó conservar PostgreSQL como fuente transaccional y considerar MongoDB solo para lecturas documentales concretas.

Contraste de dos afirmaciones:

1. La documentación oficial de MongoDB confirma que una escritura sobre **un documento** es atómica. También indica que embeber campos relacionados permite actualizarlos juntos. Esto respalda la primera afirmación como propiedad del modelo, pero **no** demuestra una mejora de velocidad: aquí no se hizo una prueba de rendimiento entre motores. Fuente: https://www.mongodb.com/docs/manual/core/write-operations-atomicity/ y https://www.mongodb.com/docs/manual/tutorial/model-data-for-atomic-operations/
2. MongoDB admite transacciones de varios documentos, por lo que sería incorrecto afirmar que no puede garantizar atomicidad entre pedido y stock. Esas transacciones deben usarse si la operación abarca documentos distintos. PostgreSQL ya agrupa el pedido y sus cambios de stock en una transacción. Fuentes: https://www.mongodb.com/docs/manual/core/transactions/ y https://www.postgresql.org/docs/17/tutorial-transactions.html

Conclusión (menos de 200 palabras). Mantendría PostgreSQL como fuente principal de FoodStore. El pedido, sus líneas y el stock tienen reglas de integridad que ya están implementadas mediante claves, triggers y un procedimiento transaccional. MongoDB resulta cómodo para consultar pedidos completos: las líneas embebidas se recuperan junto con la cabecera y la muestra migrada devolvió los mismos IDs que PostgreSQL. Sin embargo, almacenar un pedido en un único documento no vuelve atómica una actualización simultánea del stock en otros documentos. Si hubiera una necesidad concreta de lecturas documentales, usaría MongoDB como proyección de consulta sincronizada desde PostgreSQL, aceptando y controlando su posible retraso. Esta práctica demuestra equivalencia funcional en cinco pedidos; no demuestra que un motor sea más rápido que el otro.